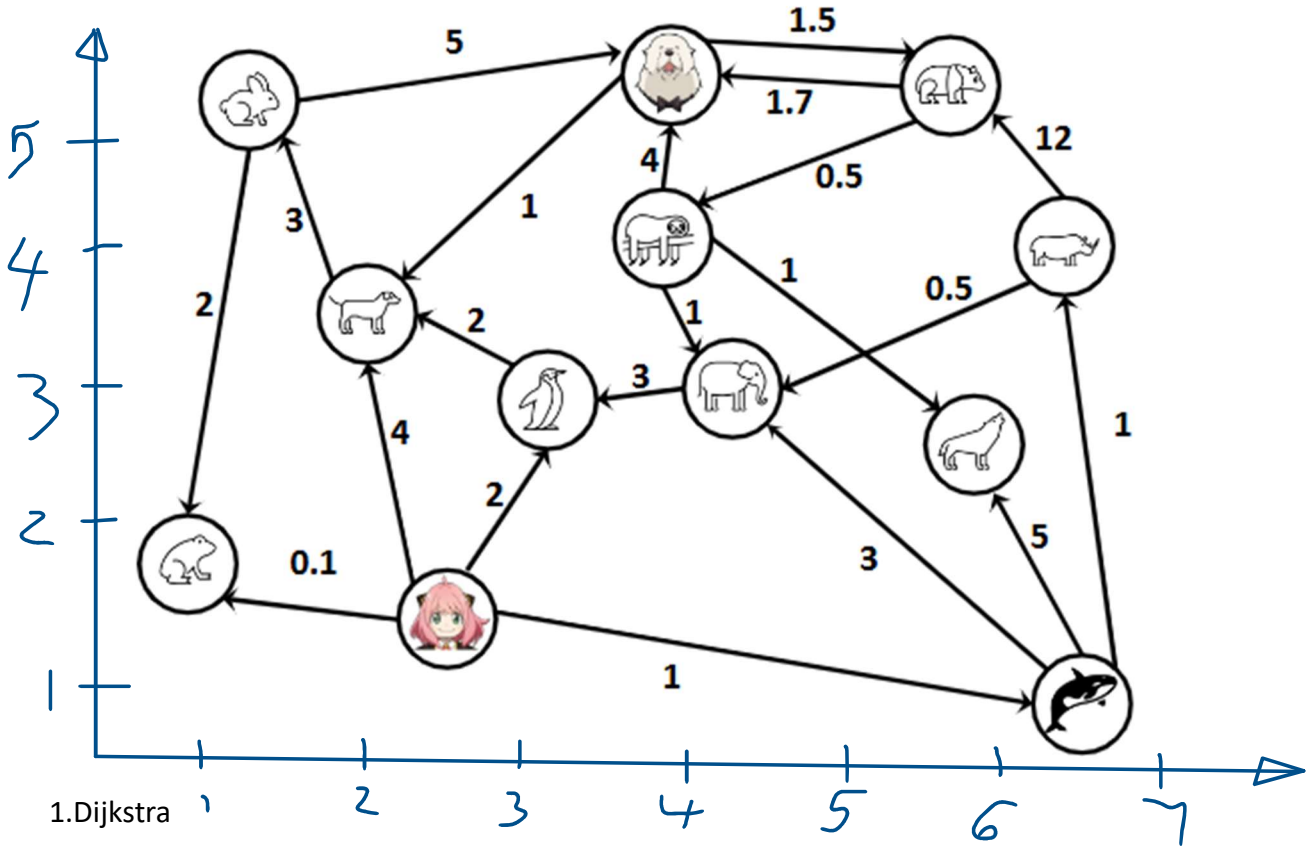
Assignment 7



1.Dijkstra

1: Anya = 0

All other nodes = infinity

PQ:- [Anya: 0]

Visited:- [Anya]

2: Anya(0): Penguin =0 + 2 = 2, Dog = 0 + 4 = 4, frog = 0 + 0.1 = 0.1, Orca = 0 + 1 = 1

PQ:-[frog: 0.1, Orca: 1, Penguin: 2, Dog: 4]

Visited:- [Anya, Frog]

3: Frog(0.1): no other vertices.

PQ:-[Orca: 1, Penguin: 2, Dog: 4]

Visited:- [Anya, Frog, Orca]

4. Orca(1): Elephant = 1 + 3 =4, Wolf = 1 + 5 = 6,Rhino = 1 + 1 = 2

PQ:-[Penguin: 2, Rhino: 2, Dog: 4, Elephant: 4, Wolf: 6]

Visited:- [Anya, Frog, Orca, Penguin]

5. Penguin(2): Dog = 2 + 2 = 4 (= 4. No change)

PQ:-[Rhino: 2, Dog: 4, Elephant: 4, Wolf: 6]

Visited:- [Anya, Frog, Orca, Penguin, Rhino]

6. Rhino(2): Elephant = 2 + 0.5 = 2.5 (> 4. Replace 4), Bear = 2 + 12 = 14.

PQ:-[Elephant: 2.5, Dog: 4, Elephant: 4, Wolf: 6, Bear: 14]

Visited:- [Anya, Frog, Orca, Penguin, Rhino, Elephant]

7. Elephant(2.5):Penguin = 2.5 + 3 = 5.5 (> 2. No change.)

PQ:-[Dog: 4, Wolf: 6, Bear: 14]

Visited:- [Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog]

8. Dog(4): Rabbit = 4 + 3 = 7. Next Shortest

PQ:-[Wolf: 6, Rabbit: 7, Bear: 14]

Visited:- [Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf]

9. Wolf(6): has no neighbors. Next shortest

PQ:-[Rabbit: 7, Bear: 14]

Visited:- [Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit]

10. Rabbit(7): Bond = 7 + 5 = 12. Next Shortest

PQ:-[Bond: 12, Bear: 14]

Visited:- [Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit, Bond]

11. Bond(12): Bear = 12 + 1.5 = 13.5 (< 14. Replace)

PQ:-[Bear: 13.5]

Visited:- [Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit, Bond]

12. Bear(13.5): Bond = 13.5 + 1.7 = 15.2(>12. No change), Sloth = 13.5 + 0.5 = 14

PQ:-[Sloth: 14]

Visited:- [Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit, Bond, Sloth]

13. Sloth(14): Elephant = 14 + 1 = 15(> 2. No change)

PQ:-[]

Visited:- [Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit, Bond, Sloth]

2. A*

coordinates: Anya(2.5,1.5) | frog(1, 1.9) | Orca(6.5, 1) | Dog(2, 3.5) | Penguin(3, 3) | Elephant(4.5, 3.1) | Wolf(6, 2.5) | Sloth(4, 4) | Rhino(6.5, 4) | Rabbit(1.3, 5.5) | Bond(4.2, 5.7) | Bear(6, 5.6)

H(n) = $\sqrt{(x_1 - x_2) + (y_1 - y_2)}$

Anya to frog:

[Frog: h =0, g = 0.1, f = 0.1] [Dog: h =1.89, g = 4, f= 5.89] [Peng: h = 2.28, g = 2, f= 4.28] [Orca: h= 5.57, g = 1, f = 6.57]

Anya to Orca:

[Frog: h =5.57, g = 0.1, f = 5.67] [Dog: h =5.15, g = 4, f= 9.15] [Peng: h = 4.03, g = 2, f= 6.03] [Orca: h= 0, g = 1, f = 1]

Anya to Penguin:

[Frog: h =2.28, g = 0.1, f = 2.38] [Dog: h =1.12, g = 4, f= 5.12] [Peng: h = 0, g = 2, f= 2] [Orca: h= 4.03, g = 1, f = 5.03]

Anya to Dog:

[Frog: h =1.89, g = 0.1, f = 1.99] [Dog: h =0, g = 4, f= 4] [Peng: h = 1.12, g = 2, f= 2.12] [Orca: h= 5.15, g = 1, f = 6.15]

- From Penguin

  [Dog: h =0, g = 2 + 2 = 4, f= 4]

Anya to Elephant:

[Frog: h =3.70, g = 0.1, f = 3.80] [Dog: h =2.53, g = 4, f= 6.53] [Peng: h = 1.50, g = 2, f= 3.50] [Orca: h= 2.90, g = 1, f = 3.90]

- Fast forwarding because Penguin has no path to elephant, go to next smallest f(n)
- From Anya
  [Frog: h =3.70, g = 0.1, f = 3.80] [Dog: h =2.53, g = 4, f= 6.53][Orca: h= 2.90, g = 1, f = 3.90]
- No path from Frog, next smallest f(n)
- From Anya
  [Dog: h =2.53, g = 4, f= 6.53][Orca: h= 2.90, g = 1, f = 3.90]
- From Orca
  [Elephant: h = 0, g = 4, f = 4][Wolf: h = 1.62, g = 6, f = 7.62][Rhino: h = 2.19, g =2, f = 4.19]

Anya to Wolf:

[Frog: h =5.04, g = 0.1, f = 5.14] [Dog: h =4.12, g = 4, f= 8.12] [Peng: h = 3.04, g = 2, f= 5.04] [Orca: h= 1.58, g = 1, f = 2.58]

- From Orca
  [Elephant: h = 1.62, g = 4, f = 5.62][Wolf: h = 0, g = 6, f = 6][Rhino: h = 1.58, g =2, f = 2.58]

Anya to Rabbit:

Anya to Bond:

Anya to Bear:

Anya to Sloth:

Anya to Rhino:

3. A* will find H in less iterations than Dijkstra. A*'s algorithm minimizes the nodes to be traversed by using heuristics to look ahead and estimate the cost from the visited node to the target, making the traversal more precise. Dijkstra traverses the graph blindly until all nodes' paths are accounted for or it reaches the target.

7. Asymptotic complexity:

Draw Graph:

Time: O(V^2), where v is the number of vertices represented in the string. The buildGraph

function uses loops that iterate for each vertex to assign default 0 values to the matrix and then assign adjacent values to each row. So, setting up the adjacency matrix gets done in O(V) time. However, at the end, the function calls to a drawGraph function. This unction has 3 loops to print the matrix the format. The first loop checks all strings for the longest string's length for formatting the output, the second loop prints each vertex for the column labels, and the third loop consists of nested loops; The outer loop prints a vertex's label, then the inner loop prints out the corresponding adjacency array. These loops combine perform at a complexity of O(V) + O(V) + O(V^2) -> O(V^2). Therefore, the complexity of buildGraph is O(V^2).

The space complexity is O(V^2). The buildgraph function creates a matrix of size V * V to store the adjacency values for each vertex and a string array that stores the labels of each vertex. All other variables created – out1, out2, maxLength, format, numFormat, I, and j - store constant values.

Weakly Connected:
Time: O(V^4) where V represents the number of vertices in the graph. The isWeaklyConnected function uses nested loops that both iterate V times to build an undirected matrix from the original vertex. It also uses an additional nested loop statement to perform a BFS between all nodes, and the BFS performs at O(V^2) in the worst case. That makes the time O(V^2) + O(V * V * V^2) -> O(V^4).
Space: O(V^2). The isWeaklyConnected creates another matrix that stores an undirected version of the original input matrix. The BFS creates a queue and an array. The array stores a Boolean value that tracks the visited status of each corresponding vertex. In the worst case for the queue, at least one of the vertices could be adjacent to all other nodes, pushing all of them into the queue. But with the undirected matrix having the complexity of the highest degree, the space for this algorithm is O(V^2).

Cycles of 4:
Time complexity: O(V^4), where V represents the number of nodes represented in the graph, as the function consists of 4 nested loops and all loops iterate from 0 to V – 1. The space complexity is O(1), as only one variable – n - is created in the function and stores constant memory for the integer value of the matrix length.